

Laboratory Activity No. 1 Real-Time Multisensor Room Monitoring System

Name: Cassandra Khan P. Mendoza	Date: September 27, 2026
Section: B187.1	Repository: github.com/monxx-ie/BCA182-freetos-multisensor

1. Problem and Requirements

For this lab I made a room monitor on an STM32 Blue Pill using PlatformIO, the STM32Cube HAL and FreeRTOS. It runs in the Wokwi simulator. The device reads temperature, humidity, light and motion, shows one of them on an OLED, sounds a buzzer if the temperature is too low or too high, and turns the screen off when nobody is around.

I used FreeRTOS because these jobs happen at different speeds. The sensors only need to be read every 2 seconds, but the encoder and the PIR should respond right away. Putting each job in its own task made this easier to handle than one big loop.

ID	Requirement	How I did it
FR-01	Temperature	SensorTask reads the DHT22 every 2 s
FR-02	Humidity	SensorTask reads the DHT22 every 2 s
FR-03	Ambient light	SensorTask reads the ADC and converts it to 0-100 %
FR-04	Motion	MotionTask checks the PIR every 100 ms
FR-05	OLED display	DisplayTask shows one reading at a time
FR-06	Encoder navigation	InputTask changes the page in both directions
FR-07	Temperature alarm	AlarmTask turns on the buzzer below 18 C or above 30 C
FR-08	Activity state	StateTask keeps track of ACTIVE / INACTIVE
FR-09	Auto inactive	No motion for 15 s makes the system INACTIVE
FR-10	Auto reactivate	Motion makes it ACTIVE again

Wokwi adaptation. Every part is a Wokwi model, so I tested by moving the sliders on the DHT22 and light sensor, clicking "Simulate motion" on the PIR, and using the encoder arrows. The serial monitor is connected to USART1. The light reading is only a relative percentage, not real lux. I also had to use a Wokwi-compatible version of FreeRTOS, which I explain in Section 7.

2. System Architecture and Design

Hardware

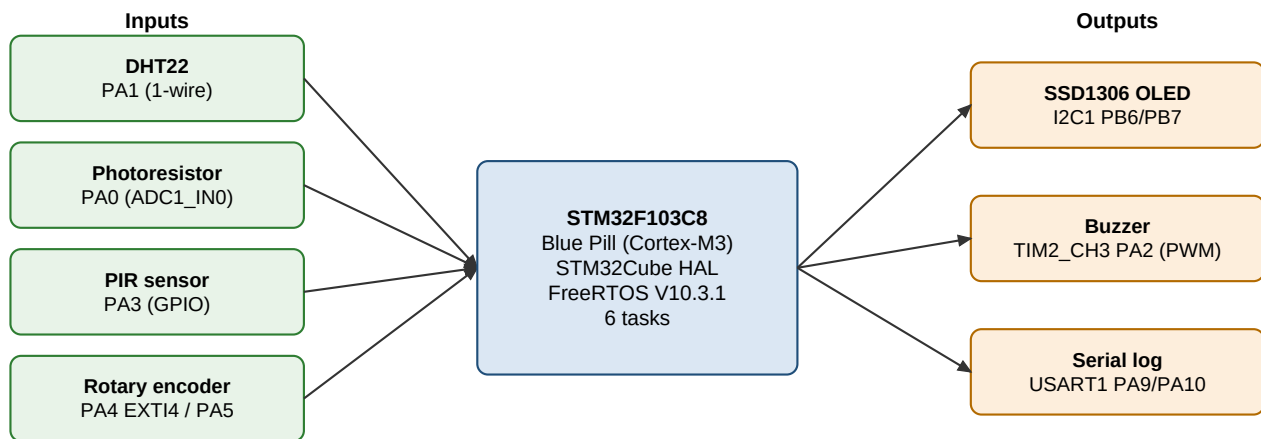


Figure 1. Hardware overview.

Part	Pin	Mode	Note
Photoresistor	PA0	ADC1 channel 0	PA0-PA7 can be used as ADC
DHT22	PA1	Output for the start pulse, then input with pull-up	I moved it from PA0 so the light sensor could use the ADC
Buzzer	PA2	TIM2 PWM, 1 kHz	Moved from PB0 because PB0 is on TIM3, which the RTOS uses
PIR	PA3	Input, pull-down	Low when there is no motion
Encoder	PA4 / PA5	Interrupt on CLK / input on DT	
Serial	PA9 / PA10	USART1, 115200	Wokwi serial monitor
OLED	PB6 / PB7	I2C1, 400 kHz	
LED	PC13	Output	Turns on if an assert fails

Timers: TIM2 for the buzzer, TIM3 for the FreeRTOS tick, and TIM4 for the HAL tick (set up with STM32CubeMX).

Software

I split the code into files so main.c would not have everything in it:

Group	Files	What they do
Start-up	main.c, app.c	main() sets up the hardware and calls app_main(), which makes the RTOS objects and tasks and starts the scheduler
Tasks	sensors.c, display.c, input.c, alarm.c, motion.c, system_state.c	One task per job
Logic	logic.c	Alarm, page and state decisions. No hardware code, so I could unit test it on my PC
Drivers	dht22.c, ldr.c, ssd1306.c, buzzer.c, log.c	Talk to the hardware
RTOS setup	rtos_objects.c, rtos_hooks.c, FreeRTOSConfig.h	Queues, event group, mutex and FreeRTOS hooks
Generated	stm32f1xx_hal_msp.c, stm32f1xx_it.c, stm32f1xx_hal_timebase_tim.c	Made by STM32CubeMX

State machine

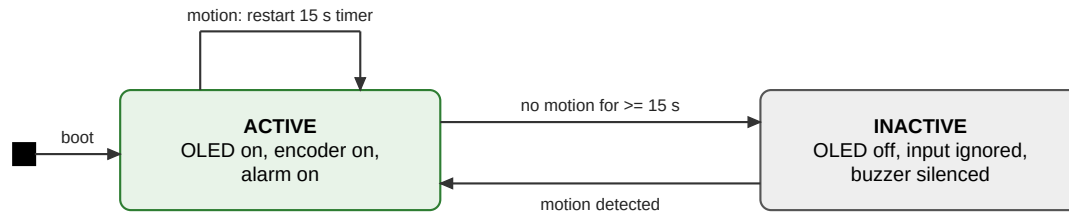


Figure 2. ACTIVE / INACTIVE state machine (evaluateSystemState() in logic.c).

When ACTIVE, the OLED is on, the encoder works and the alarm can sound. When INACTIVE, the OLED is turned off and DisplayTask waits on the ACTIVE bit, so it doesn't draw anything. Encoder turns are ignored and the buzzer is kept off. MotionTask keeps running in both states because motion is what wakes the system up.

3. FreeRTOS Architecture

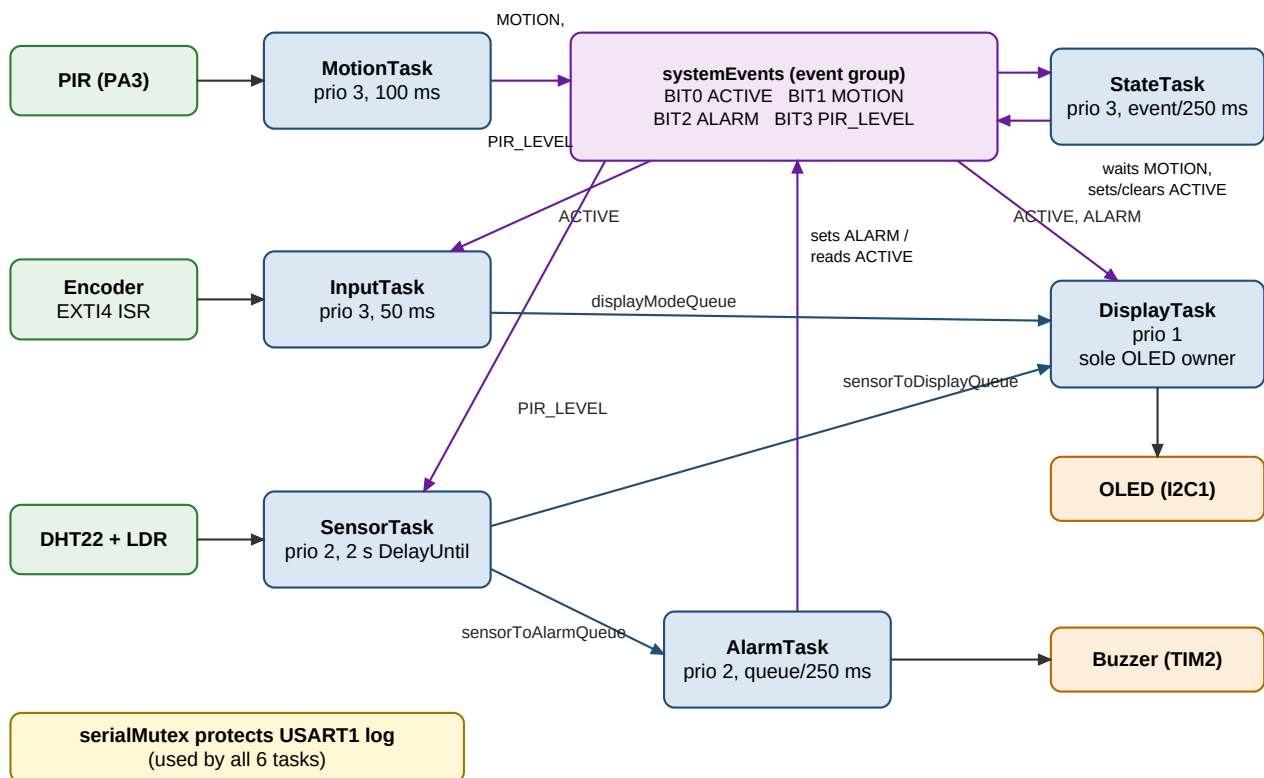


Figure 3. How the tasks talk to each other. Blue = queues, purple = event group bits.

Task	Job	Trigger / period	Priority	IPC	Usually blocked on
MotionTask	Read PIR	100 ms	3	Sets MOTION and PIR_LEVEL bits	vTaskDelay
InputTask	Encoder to page	50 ms	3	displayModeQueue; reads ACTIVE	vTaskDelay
StateTask	ACTIVE / INACTIVE	Motion bit or 250 ms	3	Waits for MOTION; sets ACTIVE	xEventGroupWaitBits
SensorTask	DHT22 + light	2 s	2	Writes both sensor queues	vTaskDelayUntil
AlarmTask	Temperature check, buzzer	New reading or 250 ms	2	sensorToAlarmQueue; sets ALARM	xQueueReceive

Task	Job	Trigger / period	Priority	IPC	Usually blocked on
DisplayTask	Draw OLED	New data or 100 ms	1	Two queues + event group	xQueueReceive

Priorities. I gave MotionTask, InputTask and StateTask priority 3 because they react to a person. If motion is handled late, the screen stays off while someone is in the room, and a slow encoder feels broken. They only do a tiny bit of work each time, so they don't take much time from the other tasks. SensorTask and AlarmTask are priority 2 because they run every 2 seconds and a few milliseconds of delay doesn't matter for room temperature. DisplayTask is priority 1 because sending a full frame to the OLED is the slowest thing in the program and it's fine if it happens a little late. If DisplayTask was higher, a redraw could delay the encoder (see fault experiment 2).

Task states. A task is Running only while it is doing its work, like reading a sensor. While it waits in vTaskDelay, vTaskDelayUntil, xQueueReceive or xEventGroupWaitBits it is Blocked and doesn't use the CPU. It is Ready when it could run but a higher priority task is running. I saw this in Part III: Task A (priority 1) and Task B (priority 2) woke up on the same tick and Task B always printed first while Task A waited in Ready. When all tasks are blocked the idle task runs and sleeps with __WFI(). I didn't use Suspended or Deleted because all my tasks run for the whole time.

vTaskDelayUntil vs vTaskDelay. vTaskDelay(2000) waits 2000 ms starting from when it is called, which is after the sensors were already read. So one cycle is really 2000 ms plus the time the read took, and this extra time adds up every cycle, so the readings slowly drift. vTaskDelayUntil waits until a fixed time (last wake time + 2000 ms), so SensorTask wakes at 0, 2, 4 s no matter how long the DHT22 read takes.

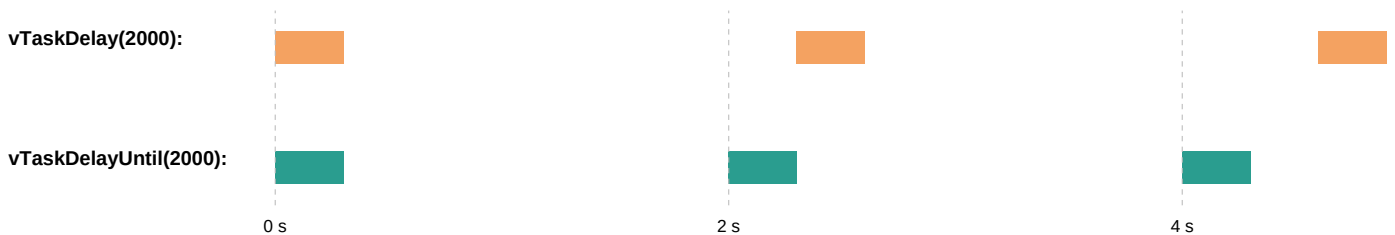


Figure 4. With a 300 ms read, vTaskDelay drifts but vTaskDelayUntil stays on a 2 s schedule.

Queues.

Queue	From / to	Data
sensorToDisplayQueue	SensorTask to DisplayTask	SensorData
sensorToAlarmQueue	SensorTask to AlarmTask	SensorData
displayModeQueue	InputTask to DisplayTask	DisplayMode

SensorData has the temperature, humidity, light level, motion and a flag that says if the DHT22 read worked. I used queues instead of global variables because the queue copies the whole struct at once. With a global, DisplayTask could read it while SensorTask is halfway through writing it and get a new temperature with an old humidity. The queue also wakes up the waiting task when data arrives, so it doesn't have to keep checking. Each queue only holds 1 item and I use xQueueOverwrite, so the task always gets the newest reading. I needed two sensor queues because reading from a queue removes the item, so DisplayTask and AlarmTask would take readings from each other.

Event group.

Bit	Name	Set / cleared by	Read by	Meaning
0	EVENT_ACTIVE	StateTask	DisplayTask, InputTask, AlarmTask	System is ACTIVE
1	EVENT_MOTION	Set by MotionTask, cleared by StateTask	StateTask	Motion happened
2	EVENT_ALARM	AlarmTask	DisplayTask	Alarm is on

Bit	Name	Set / cleared by	Read by	Meaning
3	EVENT_PIR_LEVE L	MotionTask	SensorTask	Current PIR output

I used an event group because the ACTIVE state has to be seen by three tasks at once, and a queue can only give each item to one task. EVENT_MOTION is cleared when StateTask reads it, so each motion is only counted once.

Mutex. The shared resource is USART1, the serial port for log messages. All six tasks print to it, and sometimes at almost the same time (for example a page change, a motion message and an alarm change in the same second). HAL_UART_Transmit sends one character at a time, so if the scheduler switched tasks in the middle, the characters from two messages would mix together into one unreadable line. The mutex makes sure only one task prints at a time. I used a recursive mutex so Log_Begin() can lock once for a line that is made of several Log() calls. A mutex also has priority inheritance, so if DisplayTask (priority 1) holds it while a priority 3 task is waiting, DisplayTask gets boosted and finishes quickly. The other place a race could happen is the encoder counter, which the interrupt changes and InputTask reads, so InputTask reads and clears it inside a short critical section.

4. Implementation

Start-up. main() sets the vector table address, calls HAL_Init(), sets up the LED, UART, sensors, encoder and PIR, and then calls app_main(). app_main() creates the queues, event group and mutex, creates the six tasks and starts the scheduler. If anything fails it prints an error and stops.

DHT22. The DHT22 needs timing in microseconds, so I used the DWT cycle counter. The start pulse is sent with the pin as an output, then the pin is changed to input with pull-up to read the reply. At first every read failed because I used open-drain output, and on the STM32F1 the pull-up only works in input mode. The reply is read inside a critical section so nothing interrupts the timing. Every wait loop has a timeout so it can't get stuck. If the checksum is wrong, the reading is marked invalid and the last good values are kept.

Light sensor. The ADC gives 0-4095. The module's voltage goes up when it's darker, so I flip it: $\text{light \%} = (4095 - \text{raw}) \times 100 / 4095$. I checked that brighter gives a higher number.

OLED. I wrote a small SSD1306 driver with a 5x7 font and a frame buffer. Only DisplayTask uses it, and it only redraws when the data, page or alarm changes.

Encoder. The CLK pin triggers an interrupt that just adds or subtracts 1 from a counter depending on DT. InputTask reads the counter every 50 ms and calls nextDisplayMode() or previousDisplayMode().

Alarm. evaluateTemperature() returns NORMAL, LOW_TEMPERATURE or HIGH_TEMPERATURE. Exactly 18 C and 30 C count as normal. AlarmTask uses it on every valid reading and turns the TIM2 PWM on or off. The buzzer is always off when INACTIVE.

State machine. evaluateSystemState() decides the next state from the current state, whether there was motion, and how long since the last motion. StateTask waits for the motion bit with a 250 ms timeout so it can also check the 15 s timeout.

C instead of C++. The lab examples use enum class and .cpp files, but I used C with typedef enum because the STM32 HAL and the CubeMX files are in C. I still used named enums (DisplayMode, AlarmState, SystemState) instead of plain numbers.

5. Verification and Testing

Unit tests

I wrote 13 unit tests for logic.c and ran them on my PC with `pio test -e native`. All 13 passed (saved in docs/unit-test-results.txt).

Group	Test	Expected
Alarm	17.9 C	LOW_TEMPERATURE
Alarm	18.0 C	NORMAL
Alarm	25.0 C	NORMAL
Alarm	30.0 C	NORMAL
Alarm	30.1 C	HIGH_TEMPERATURE
Navigation	next from Temperature	Humidity
Navigation	next from Motion	Temperature (wraparound)
Navigation	previous from Humidity	Temperature
Navigation	previous from Temperature	Motion (wraparound)
State	ACTIVE, no motion, 10 s	ACTIVE
State	ACTIVE, no motion, 15 s	INACTIVE
State	INACTIVE, no motion	INACTIVE
State	INACTIVE, motion	ACTIVE

Wokwi functional tests

ID	What I did	Expected	What I saw	Result
FT-01	Temperature slider to 43.7 then 58.6 C	Temperature updates	OLED and log showed 43.7 then 58.6 C	PASS
FT-02	Humidity slider to 73.0 then 56.5 %	Humidity updates	Humidity page followed	PASS
FT-03	Moved the lux slider	Light changes	Went up when brighter, down when darker (75 % at default)	PASS
FT-04	Encoder clockwise	Next page	Temperature, Humidity, Light, Motion, Temperature	PASS
FT-05	Encoder counterclockwise	Previous page	Temperature to Motion, then Light	PASS
FT-06	Temperature to 50.1 C	Alarm on	HIGH_TEMPERATURE, buzzer on, '! TEMP ALARM'	PASS
FT-07	Temperature to 24.6 C	Alarm off	NORMAL, buzzer off	PASS
FT-08	Simulate motion while ACTIVE	Stays ACTIVE	'Motion detected', timer restarted	PASS
FT-09	No motion for 15 s	INACTIVE	'System INACTIVE', 'OLED off'	PASS
FT-10	Simulate motion while INACTIVE	ACTIVE again	'System ACTIVE', 'OLED on', showed 25.0 C	PASS

I also checked that while INACTIVE, turning the encoder and going above 30 C did nothing. The first DHT22 read after start-up sometimes fails because the sensor isn't ready yet, but the next reads work and the alarm ignores the bad one.

Fault experiments

I did each one separately, then put the code back with git checkout.

Experiment	What I changed	What I saw	Why
1. No blocking	Removed vTaskDelay from MotionTask	Only the MotionTask message printed. The OLED stayed on '...', no sensor data, and it never went INACTIVE.	MotionTask never blocks and has the highest priority, so it keeps the CPU and every other task starves. The idle task never runs either. In this port tasks only switch when they block, so even the other priority 3 tasks were stuck. On a normal port they would share time, but priority 1 and 2 tasks would still starve.

Experiment	What I changed	What I saw	Why
2. Higher priority	DisplayTask from 1 to 4	Nothing looked different. The encoder, PIR and alarm still worked.	DisplayTask is waiting most of the time, and priority only matters when a task is Ready. But during a redraw it would now block the encoder and motion tasks, so with a busier display it would feel laggy. I put it back to 1.
3. No mutex	Made MutexUsable() return 0	No mixed lines, even when I turned the encoder, triggered the PIR and crossed 30 C at the same time.	In this port a task only switches when it blocks, and HAL_UART_Transmit doesn't block, so a line never gets cut. With a normal port the tick could switch tasks in the middle of a line, so I kept the mutex.

6. Static Code Analysis

I ran `pio check (cppcheck)`. The first run had 0 high, 0 medium and 58 low findings. After fixing and reviewing them, the final run had 6 low (`docs/pio-check-report.txt` and `pio-check-report-after.txt`).

Finding	File / line	Count	Cause	What I did
unusedFunction	Task files, drivers, logic.c, app.c, stm32f1xx_it.c, hal_msp.c	51	cppcheck checks one file at a time, so it can't see that tasks are called through xTaskCreate, interrupts through the vector table, and callbacks by HAL/FreeRTOS	Not a real bug. Suppressed in platformio.ini with a comment
unsignedLessThanZero	dht22.c:68	1	The timeout check mixed a hardware register with unsigned math. I rewrote it with an 'elapsed' variable but the warning moved with it, so it's a false positive	Rewrote the code, then added an inline suppress with a comment
constParameterPointer	rtos_hooks.c:20, 32	2	These function signatures are fixed by FreeRTOS and HAL	Kept
constParameterPointer	stm32f1xx_hal_msp.c	4	CubeMX generated callbacks with fixed signatures	Kept

None of the findings were real bugs, but it made me clean up the DHT22 timing code.

7. Engineering Discussion

Getting FreeRTOS to run in Wokwi

This took most of my time. The code built and printed the start-up messages, but after `vTaskStartScheduler()` no task ran and Wokwi slowed down to about 6 %. To find out why, I made `configASSERT` print the file and line directly to the UART registers and added a `HardFault` handler. I also moved the HAL tick to TIM4 using CubeMX so `SysTick` was only used by FreeRTOS.

Try	Setup	What happened	What it meant
1	FreeRTOS V11 ARM_CM3	Assert at port.c:336	Wokwi doesn't store interrupt priority bits (they read as 0), so BASEPRI masking can't work
2	V11 ARM_CM0	Assert at port.c:901	The SVC call that starts the first task doesn't work right in Wokwi
3	V10.6.2 ARM_CM0	Tasks printed once. The tick counted up to about 928 then stopped	It froze when switching back to a task that was paused by PendSV
4	Wokwi-compatible port from classmate Ni-ear	Task A and B repeat every second	It uses TIM3 for the tick and switches tasks without SysTick/SVC/PendSV
5	Same, after adding the DHT22	Froze after a few readings, especially when I changed the sliders	Two more bugs in the port (below)

The first bug was that the port still used BASEPRI to block interrupts, which does nothing in Wokwi, so the TIM3 tick could interrupt FreeRTOS while it was changing its task lists. I changed `portmacro.h` to use `PRIMASK (cpsid i / cpsie i)`. After that it froze completely, because the port kept one shared critical section

counter. FreeRTOS can switch tasks inside a critical section, so the next task got the wrong count and interrupts stayed off. I changed the context switch to save the counter for each task. After both fixes it ran without problems.

Other problems

Problem	Cause	Fix
'expected declaration at end of input'	Missing } in main()	Added it
Multiple definition of SysTick_Handler	main.c and the port both had one	Kept only one
Error in FreeRTOSConfig.h	Including stm32f1xx.h caused a circular include	Only declared SystemCoreClock
DHT22 always failed	Pull-up doesn't work in output mode on STM32F1	Switch to input with pull-up to read
Buzzer pin conflict	PB0 is on TIM3, used by the RTOS tick	Moved to PA2 (TIM2)
Unit tests wouldn't build	No gcc on Windows	Installed WinLibs with winget

Trade-offs

I checked the PIR every 100 ms instead of using an interrupt because it's simpler and motion is slow anyway. On real hardware an interrupt with a task notification would react faster. I used an event group for ACTIVE because three tasks need to see it. The queues only keep the newest reading, which is fine since only the current room condition matters. The port uses a 20 Hz tick to keep Wokwi from slowing down, which is enough for this project but not for precise timing.

Limitations

The FreeRTOS port only works in Wokwi. On a real Blue Pill I would use the normal ARM_CM3 port with a 1 kHz tick, and then experiments 1 and 3 would probably show time slicing and mixed output. The light value isn't calibrated. The DHT22 read turns off interrupts for about 5 ms. The 15 s timeout is short on purpose for testing. On real hardware I would also have to deal with sensor noise and encoder bounce.

Requirements traceability

Requirement	Implementation	Verification
FR-01 Temperature	SensorTask, dht22.c	FT-01
FR-02 Humidity	SensorTask, dht22.c	FT-02
FR-03 Light	SensorTask, ldr.c	FT-03
FR-04 Motion	MotionTask	FT-08, FT-10
FR-05 OLED	DisplayTask, ssd1306.c	FT-01 to FT-05
FR-06 Encoder	InputTask, nextDisplayMode(), previousDisplayMode()	4 navigation unit tests, FT-04, FT-05
FR-07 Alarm	AlarmTask, evaluateTemperature(), buzzer.c	5 alarm unit tests, FT-06, FT-07
FR-08 Activity state	StateTask, EVENT_ACTIVE	State unit tests, FT-08 to FT-10
FR-09 Auto inactive	StateTask, evaluateSystemState()	State unit test, FT-09
FR-10 Auto reactivate	MotionTask to StateTask via EVENT_MOTION	State unit test, FT-10

8. Conclusion

The system meets all 10 requirements. It has six tasks with priorities based on how fast they need to respond, three queues for data, an event group for the system state, a mutex for the serial port, and vTaskDelayUntil for the sensor timing. All 13 unit tests and all 10 Wokwi tests passed, and static analysis didn't find any real bugs.

The biggest things I learned came from when things broke. A task that never blocks takes over the CPU, priority only matters when a task is Ready, and the code shouldn't depend on the scheduler switching at a lucky time. When the system froze with no error, printing the assert location and the tick count helped a lot

more than guessing.

If I did this again I would test it on a real Blue Pill with the normal FreeRTOS port, use interrupts and task notifications for the encoder and PIR, calibrate the light sensor, and show more information on the OLED.

References

BCA182 Laboratory Activity No. 1, Asst. Prof. Paul Rodolf P. Castor, MSU-IIT. FreeRTOS Kernel V10.3.1 (github.com/FreeRTOS/FreeRTOS-Kernel). Wokwi-compatible FreeRTOS port by classmate Ni-ear

(github.com/Ni-ear/bca182-freertos-multisensor); I added the PRIMASK and critical-nesting fixes.

STM32CubeF1 HAL and STM32CubeMX. PlatformIO, Unity and cppcheck. Wokwi docs (docs.wokwi.com).

Public 5x7 font. I used Claude (Anthropic) as an assistant for debugging and writing; I built, ran and tested everything myself.